

CAP455: OBJECT ORIENTED PROGRAMMING USING C++

The Proposal For

Rural Irrigation Turn Allocation

Master of Computer Applications (MCA)

Academic Session: 2026–2027

Submission: CA1 Proposal

Submitted By:

Nishant Kumar

Reg No.: 12623156

Roll No.: RD1P2635A05

Submitted To:

Mr. Adarsh Kumar Singh

GitHub Repo. Link.: https://github.com/nishantsin512/Rural-Irrigation-Turn-Allocation

TABLE OF CONTENTS

Content	Page Number
1. EXECUTIVE SUMMARY	3
2. PROBLEM STATEMENT & PROPOSED SOLUTION	4
3. ENTITY & MAIN CLASS DESIGN	5-6
4. UNIQUE KEY & DUPLICATE PREVENTION	7
5. DATA DICTIONARY: CLASS ATTRIBUTES	8-9
6. DATA DICTIONARY: FUNCTIONS	10
7. CONCEPT MAPPING	11-12
8. UML CLASS DIAGRAM	13
9. SYSTEM FLOWCHART	14
10. SOURCE CODE	15-18
11. OUTPUT	19

1. EXECUTIVE SUMMARY

Rural Irrigation Turn Allocation is a menu-driven console application developed in C++ to organize basic farmer registration and irrigation scheduling information. In rural agricultural communities, several farmers may depend on shared water sources such as canals and wells. They need a clear record of who is registered, which crop is being cultivated, and when a farmer is scheduled to receive irrigation. When these details are recorded manually in notebooks or separate spreadsheets, records can become difficult to maintain as the number of farmers increases. Searching for a particular farmer may take time, duplicate identification numbers can be entered, and incomplete or inconsistent information may cause confusion during allocation. The CA1 application demonstrates a basic computerized approach to this problem using Object-Oriented Programming concepts. Its single entity class, Farmer, stores a farmer's identification number, name, village, land area, crop type, water source, irrigation date, starting hour, duration and status. Private data members keep the information within the class, while public member functions provide controlled input, validation and display operations. A static data member counts successfully registered farmers, and a static function returns that count. An inline getter returns a farmer's ID, while a friend function displays a concise farmer summary. The main function creates a fixed-size array of 50 Farmer objects and uses an object pointer to traverse the array. A do-while loop repeatedly displays a switch-case menu. The available operations are farmer registration, display of all registered farmers, search by ID, display of the total count and exit. Before a new record is accepted, the program checks that the ID is positive and compares it with IDs already stored in the array. It also validates the required text fields, land area and irrigation timing. The application uses std::string for textual information and the this pointer inside member functions to refer to the current object. This CA1 version stores records only in memory, so the data is available while the program is running and is not retained after exit. Later phases are planned to extend the same project with file persistence, inheritance, operator overloading, polymorphism, exception handling and standard library containers.

2. PROBLEM STATEMENT & PROPOSED SOLUTION

2.1 Background and Problem Statement

Irrigation is an important part of agricultural activity, particularly in areas where farmers depend on shared water resources. A canal, community well or other source may serve multiple farmers, and each farmer may require water at a particular date and time. For this reason, the basic information about farmers and their planned irrigation periods needs to be recorded in a consistent manner. When records are maintained manually, the responsibility for keeping information accurate rests entirely on the person writing or updating the register.

One difficulty with paper-based records is the possibility of duplicate entries. A farmer may be registered more than once if an earlier entry is not checked carefully. Duplicate identification numbers can make it unclear which record should be used when searching for a farmer. The same issue can occur when information is copied between notebooks or spreadsheets. If different people maintain separate copies, the records may not match, and a user may rely on an outdated version. A simple identification check is therefore useful even in a small record-management application.

Manual searching and record maintenance can also consume time. A user who needs to find a farmer's village, land area or crop type may have to inspect several pages or spreadsheet rows. Irrigation details such as water source, date, starting hour and duration must also be available in a clear format. Physical registers may be misplaced, damaged or become difficult to read. A spreadsheet may be accidentally overwritten or saved in more than one version. These problems can reduce confidence in the information and make routine administration less organized.

2.2 Proposed Solution

The proposed CA1 solution is a C++ console application that stores farmer and irrigation information in an organized object. The Farmer class combines related details into one entity and provides member functions for accepting, validating and displaying information. The use of private data members supports encapsulation, while public functions provide a controlled way to work with the data. A fixed-size array stores up to 50 Farmer objects during one program run.

The menu allows the user to register a farmer, display all registered farmers, search by ID and view the total registered count. Before a new record is accepted, the program checks whether the entered ID is positive and compares it with the IDs of existing records. The validation function checks required fields and irrigation timing. This approach does not replace a full village water-management platform, but it demonstrates a structured first step using the C++ concepts required for CA1.

2.3 Project Objectives

- To represent farmer and irrigation information through a C++ entity class named Farmer.
- To provide a menu-driven method for creating, displaying and searching farmer records in memory.
- To prevent duplicate farmer IDs by checking existing array objects before accepting a new record.
- To validate important fields such as name, village, land area, crop type, water source and irrigation timing.
- To demonstrate CA1 concepts including static members, inline and friend functions, arrays of objects, object pointers, the this pointer, strings and reference parameters.

3. ENTITY & MAIN CLASS DESIGN

3.1 Entity Overview

The CA1 implementation uses one main entity class named Farmer. The class represents a registered farmer together with the irrigation details recorded for that farmer in this version of the application. Combining these details in one entity keeps the first-phase program straightforward and allows the required array-of-objects and class concepts to be demonstrated without introducing inheritance or separate scheduling classes.

3.2 Farmer Class Attributes

Attribute	Type	Description
farmerId	int	Unique identification number assigned to a farmer.
name	string	Full name of the farmer.
village	string	Village in which the farmer is recorded.
landArea	double	Agricultural land area measured in acres.
cropType	string	Crop cultivated by the farmer.
waterSource	string	Water source entered for the irrigation record, such as canal or well.
turnDate	string	Date entered for the irrigation schedule.
startHour	int	Starting hour of the irrigation period.
durationHours	int	Duration of irrigation in hours.
status	char	Simple status marker set to A for an accepted record.
farmerCount	static int	Class-level counter for successfully registered records.

3.3 Responsibilities of the Class

The Farmer class is responsible for initializing an unused array element through the setDefaults() member function, accepting the record through inputDetails(int id), checking the entered values through isValid(), and displaying the record through displayDetails(). The getFarmerId() function returns the ID used by the duplicate-check and search operations. The static function getFarmerCount() returns the class-level count. The friend function showFarmerSummary(const Farmer& f) displays a short summary using private data members.

3.4 Encapsulation and Access Control

All attributes are declared private. This means that code outside the class cannot directly read or change these data members using ordinary object access. The public member functions form the interface through which the main function works with a Farmer object. For example, the main function calls inputDetails() to collect information and displayDetails() to show a record. The validation function checks whether the values meet the program's basic conditions before the record is counted.

The friend function is a deliberate exception to normal private access. It is declared with the friend keyword inside the class and defined outside the class. It is not a member function, but it is allowed to access the class's private members because the class grants that permission. The const reference parameter avoids making a separate copy and prevents the friend function from modifying the referenced object through that parameter.

3.5 Scope of the CA1 Entity

This version uses one Farmer object per registered record. Irrigation information is stored as attributes of that object. It does not create an independent irrigation-turn entity or a separate turn ID in CA1. This is a deliberate simplification of the current CA1 code and should be described consistently in the proposal, UML diagram and viva. Later phases may refine the model when the course requirements and faculty feedback permit.

4. UNIQUE KEY & DUPLICATE PREVENTION

4.1 Unique Key

The variable farmerId is the unique key used to identify each Farmer record in the CA1 application. It is declared as an integer data member inside the Farmer class. A valid ID must be greater than zero. The program uses getFarmerId() to read the ID of an existing object during duplicate checking and searching.

4.2 Duplicate Checking Process

When the user selects the registration option, the program first checks whether the array has reached its maximum capacity of 50 objects. It then asks for a farmer ID and verifies that the input is numeric and positive. An integer flag is initialized to zero. The for loop starts at index zero and checks only the records counted in total. For each existing record, the program compares the new ID with the ID returned by ptr[i].getFarmerId().

If the comparison is equal, the flag is changed to one and the loop is stopped using break. After the loop, a flag value of one means that the ID is already present, so the new record is rejected. If the flag remains zero, the program calls inputDetails(id) for the next unused array element. The record is included in total only if isValid() returns true.

4.3 Pseudocode

```
START
Check whether total is less than 50
Input id
If input is invalid or id <= 0:
    Display an error
Else:
    Set flag = 0
    For i = 0 to total - 1:
        If ptr[i].getFarmerId() == id:
            Set flag = 1
            Break
    If flag == 1:
        Display duplicate ID message
    Else:
        Call arr[total].inputDetails(id)
        If arr[total].isValid():
            Increase total
        Else:
            Reset the unused array element
STOP
```

4.4 Why the Loop Uses total

The loop condition is i < total because only the first total array positions contain registered records. The remaining positions are unused capacity. Searching all 50 positions would unnecessarily inspect objects that have not been registered. The total variable therefore acts as the current number of occupied positions, while the fixed array provides the maximum capacity.

4.5 Validation Boundary

Duplicate prevention is limited to the records stored during the current execution. Since CA1 uses in-memory storage only, all records and the duplicate-check history are lost when the program exits. Persistent duplicate checking across separate program runs is a future file-handling concern and is not claimed as a feature of this CA1 version.

5. DATA DICTIONARY: CLASS ATTRIBUTES

This data dictionary describes the variables declared in the current CA1 Farmer class. The size of int, double and char shown below is typical for a common g++ environment; exact sizes can depend on the compiler and platform. The size of a std::string object is implementation-dependent and is not the same as the length of the text stored in it.

Variable Name	Data Type	Access	Typical Size	Purpose and Validation
farmerId	int	private	4 bytes	Unique farmer key; must be greater than zero.
name	string	private	Implementation-dependent	Farmer's name; must not be empty.
village	string	private	Implementation-dependent	Village name; must not be empty.
landArea	double	private	8 bytes	Land area in acres; must be greater than zero.
cropType	string	private	Implementation-dependent	Crop name; must not be empty.
waterSource	string	private	Implementation-dependent	Irrigation water source; must not be empty.
turnDate	string	private	Implementation-dependent	Irrigation date as entered by the user; must not be empty.
startHour	int	private	4 bytes	Starting hour; must be from 0 through 23.
durationHours	int	private	4 bytes	Duration; must be positive and no more than 24 hours.
status	char	private	1 byte	Status marker; set to 'A' after successful validation.
farmerCount	static int	private	4 bytes	Shared class counter incremented after a valid registration.

5.1 Local Variables Used in main()

Variable	Type	Purpose
MAX	const int	Defines the maximum number of Farmer objects as 50.
arr	Farmer[50]	Fixed-size array used for in-memory storage.
ptr	Farmer*	Points to the first element of arr and is used to traverse objects.
total	int	Number of successfully registered records currently stored.
choice	int	Stores the user's menu selection.
id	int	Temporarily stores the farmer ID entered during registration or search.
flag	int	Set to 1 when the registration ID already exists.
found	int	Indicates whether the requested farmer ID was found during search.
i	int	Loop index used to traverse the array.

5.2 Notes on Data Types

The program uses int for IDs and whole-number time values, double for land area, char for the status marker, and string for textual information. The string type is made available by including the string header.

The code does not declare a float attribute. The static int farmerCount is shared by all Farmer objects and is defined outside the class in the source file.

The local variables MAX, arr, ptr, total, choice, id, flag, found and i belong to the main function. They are not class attributes. Keeping class attributes separate from local variables makes the data dictionary easier to interpret.

6. DATA DICTIONARY: FUNCTIONS

6.1 Farmer Class Functions

Function Name	Return Type	Concept	Purpose
setDefaults()	void	Member function; this pointer	Sets the current object fields to initial values for an unused array slot.
getFarmerId()	int	Inline function; const member function	Returns the current object's farmer ID.
getFarmerCount()	int	Static member function	Returns the shared number of successfully registered records.
isValid()	bool	Encapsulation; const member function	Checks ID, text fields, land area and irrigation-time conditions.
inputDetails(int id)	void	Member function; this pointer	Accepts farmer and irrigation details and updates the current object.
displayDetails()	void	Member function; const member function	Displays the current record if its details are valid.
showFarmerSummary(const Farmer& f)	void	Friend function; reference parameter	Displays a short summary and accesses private members with friend permission.

6.2 Static Data Definition

```
int Farmer::farmerCount = 0;
```

This statement defines and initializes the static data member outside the class. The declaration inside the class describes the member, while this definition provides its storage and initial value.

6.3 Main Function Responsibilities

Operation	Main function behaviour
Initialization	Creates the array and pointer, sets total and choice, and calls setDefaults() for all 50 objects.
Register Farmer	Reads an ID, checks positivity and duplicates, accepts details, validates the record and increases total when valid.
Display All	Uses the object pointer to display every registered record.
Search by ID	Reads an ID, loops through registered objects, displays the matching record and calls the friend summary function.
Show Total	Calls Farmer::getFarmerCount() and displays the shared count.
Exit	Displays an exit message and ends the do-while menu.

6.4 Function and Scope Clarification

The current CA1 source does not define a user-written constructor or destructor. The setDefaults() function is an ordinary public member function, not a constructor, because it has a return type of void and is called explicitly from main(). The inline function is getFarmerId(). The static member function is getFarmerCount(). The friend function is showFarmerSummary().

7. CONCEPT MAPPING

7.1 Classes and Objects

A class is a user-defined type that groups data members and member functions. An object is an instance of a class. The CA1 application defines the Farmer class to represent the information maintained for one registered farmer. The class groups identification, personal, land, crop and irrigation fields with the functions that operate on them. In main(), the statement Farmer arr[50] creates a fixed-size array containing 50 Farmer objects. Each array position can hold one record.

7.2 Encapsulation

Encapsulation means combining data and the functions that work on that data within a class, while controlling access to the data. In the Farmer class, attributes are private. The public functions setDefaults(), inputDetails(), isValid() and displayDetails() provide the permitted operations. This prevents main() from directly changing private attributes and keeps the basic validation logic within the entity class.

7.3 Static Data Member and Static Function

The static data member farmerCount belongs to the class as a whole rather than to one individual object. It is incremented after inputDetails() has accepted a valid record. The static function getFarmerCount() returns this shared value. The main function calls it using the class name, as in Farmer::getFarmerCount(). The static member is initialized outside the class with int Farmer::farmerCount = 0.

7.4 Inline Function

The function getFarmerId() is declared with the inline keyword inside the Farmer class. It returns the ID of the current object. The main function uses this getter when checking whether an entered ID already exists and when searching for a record. The inline keyword requests inline treatment from the compiler, but the compiler makes the final decision about whether to substitute the function body at a call site.

7.5 Friend Function

The function showFarmerSummary(const Farmer& f) is declared as a friend inside the Farmer class and defined outside the class. It is not a member function. The friend declaration grants it access to private members, allowing it to display selected details such as farmerId, name, village and waterSource. Its const reference parameter avoids copying the whole object and prevents modification through that reference.

7.6 Array of Objects

The array arr contains 50 Farmer objects and provides fixed-size in-memory storage. The variable total records how many positions currently contain accepted farmer records. Registration uses arr[total] as the next unused position. Display and search operations process positions from zero up to total minus one. No dynamic allocation or STL container is used in this phase.

CONCEPT MAPPING

7.7 Pointers to Objects

The statement Farmer *ptr = arr; declares a pointer to a Farmer object and initializes it with the address of the first element of the array. The pointer is used with an index, for example ptr[i].getFarmerId() or ptr[i].displayDetails(). This demonstrates access to array objects through a pointer. The pointer does not allocate memory; the array itself provides the storage.

7.8 this Pointer

The this pointer is available inside non-static member functions and points to the object on which the function is operating. For example, this->farmerId = id assigns the supplied ID to the farmerId member of the current object. The input function uses this pointer for the other attributes as well. The setDefaults() function also uses it to initialize the current array element. A static function cannot use this because it is not associated with a particular object.

7.9 std::string

The application includes the string header and uses the string type for name, village, cropType, waterSource and turnDate. Text input is collected with getline() so that entries containing spaces, such as a full name or village name, can be stored as a complete line. The program uses using namespace std; so the string type can be written as string rather than std::string.

7.10 Reference Parameter

The friend function accepts const Farmer& f. The ampersand indicates a reference parameter, so f refers to the original Farmer object supplied by the caller rather than a separate copy. The const qualifier means the function cannot use this reference to modify the object. The function can read the private attributes because it has also been declared as a friend.

7.11 Call by Value and Call by Reference

The inputDetails(int id) function receives the integer ID by value. The friend summary function receives a Farmer object by constant reference. These two parameter styles demonstrate the difference between passing a simple value and referring to an existing object. Passing the object by reference avoids copying all its data members.

7.12 Loops, Selection and Validation

The menu is repeated using a do-while loop, ensuring that the menu is displayed at least once. A switch-case statement selects the requested operation. For loops traverse the fixed-size array. The duplicate check uses an integer flag initialized to zero and changes it to one when a matching ID is found. The isValid() function checks the required fields and ensures that the irrigation period does not extend beyond hour 24.

7.13 CA1 Storage Limitation

The array stores data only in memory. The records remain available while the application is running, but they are not saved to a file. Closing the application removes the records from the program's working memory. File persistence is planned for a later phase and is not included in this CA1 proposal.

8. UML CLASS DIAGRAM

CA1 design — one main entity class: Farmer

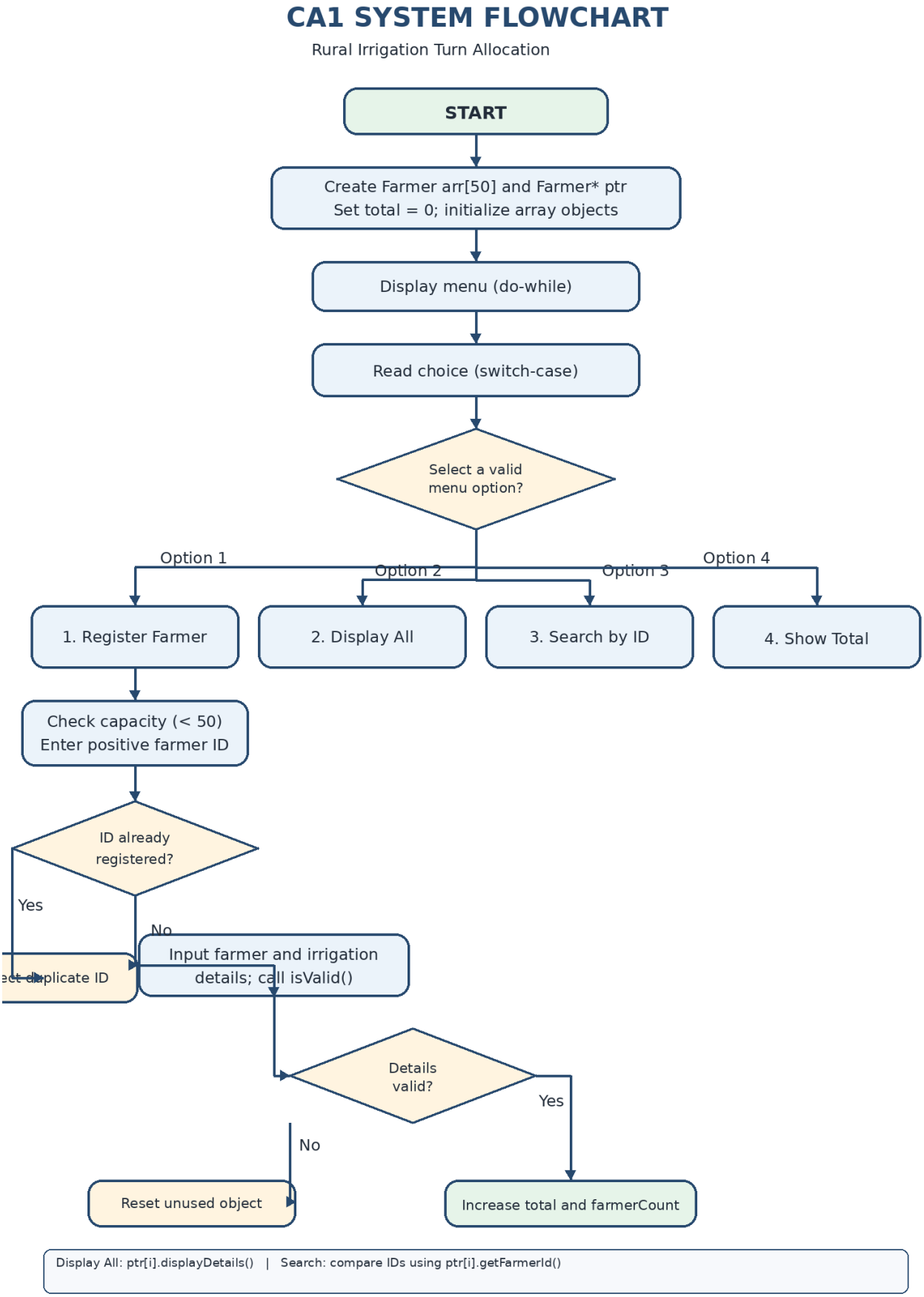
Farmer
- farmerId : int - name : string - village : string - landArea : double - cropType : string - waterSource : string - turnDate : string - startHour : int - durationHours : int - status : char - farmerCount : static int
+ setDefaults() : void + getFarmerId() : int {inline, const} + getFarmerCount() : int {static} + isValid() : bool + inputDetails(id : int) : void + displayDetails() : void + showFarmerSummary(f : const Farmer&) : void {friend}

Relationship / implementation note

main() creates a fixed array Farmer arr[50] and a Farmer* pointer named ptr to traverse it. Each array element represents one farmer record. CA1 does not define a separate IrrigationTurn class; irrigation information is stored as attributes of Farmer.

9. SYSTEM FLOWCHART

Menu navigation, farmer registration, validation and record operations



10. SOURCE CODE

File: main.cpp

```
001  #include <iostream>
002  #include <string>
003  using namespace std;
004
005  class Farmer
006  {
007  private:
008      int farmerId;
009      string name;
010      string village;
011      double landArea;
012      string cropType;
013      string waterSource;
014      string turnDate;
015      int startHour;
016      int durationHours;
017      char status;
018
019      static int farmerCount;
020
021  public:
022      void setDefaults()
023      {
024          this->farmerId = 0;
025          this->name = "";
026          this->village = "";
027          this->landArea = 0;
028          this->cropType = "";
029          this->waterSource = "";
030          this->turnDate = "";
031          this->startHour = 0;
032          this->durationHours = 0;
033          this->status = 'I';
034      }
035
036      inline int getFarmerId() const
037      {
038          return this->farmerId;
039      }
040
041      static int getFarmerCount()
042      {
043          return farmerCount;
044      }
045
046      bool isValid() const
047      {
048          if (farmerId <= 0)
049              return false;
050
051          if (name == "" || village == "" || cropType == "" || waterSource == "" || turnDate == "")
052              return false;
053
054          if (landArea <= 0)
055              return false;
056
057          if (startHour < 0 || startHour > 23)
058              return false;
059
060          if (durationHours <= 0 || durationHours > 24)
061              return false;
062
063          if (startHour + durationHours > 24)
064              return false;
065
066          if (status != 'A')
067              return false;
068
069          return true;
070      }
071
072      void inputDetails(int id)
073      {
074          this->farmerId = id;
075
076          cin.ignore(1000, '\n');
077
078          cout << "Enter Farmer Name: ";
079          getline(cin, this->name);
080
081          cout << "Enter Village Name: ";
082          getline(cin, this->village);
083
```

```
084         cout << "Enter Land Area (acres): ";
085         while (!(cin >> this->landArea) || this->landArea <= 0)
086         {
087             if (cin.fail())
088             {
089                 cin.clear();
090                 cin.ignore(1000, '\n');
091             }
092             cout << "Enter a valid positive land area: ";
093         }
094 
095         cin.ignore(1000, '\n');
096 
097         cout << "Enter Crop Type: ";
098         getline(cin, this->cropType);
099 
100         cout << "Enter Water Source (Canal/Well): ";
101         getline(cin, this->waterSource);
102 
103         cout << "Enter Irrigation Date: ";
104         getline(cin, this->turnDate);
105 
106         cout << "Enter Starting Hour (0-23): ";
107         while (!(cin >> this->startHour) || this->startHour < 0 || this->startHour > 23)
108         {
109             if (cin.fail())
110             {
111                 cin.clear();
112                 cin.ignore(1000, '\n');
113             }
114             cout << "Enter a valid starting hour (0-23): ";
115         }
116 
117         cout << "Enter Irrigation Duration (1-24 hours): ";
118         while (!(cin >> this->durationHours) ||
119                this->durationHours <= 0 || this->durationHours > 24 ||
120                this->startHour + this->durationHours > 24)
121         {
122             if (cin.fail())
123             {
124                 cin.clear();
125                 cin.ignore(1000, '\n');
126             }
127             cout << "Enter a valid duration that ends by hour 24: ";
128         }
129 
130         this->status = 'A';
131 
132         if (isValid())
133         {
134             farmerCount++;
135             cout << "\nFarmer Record Added Successfully!\n";
136         }
137         else
138         {
139             cout << "\nInvalid Farmer Record. Registration Cancelled.\n";
140         }
141     }
142 
143     void displayDetails() const
144     {
145         if (isValid())
146         {
147             cout << "\n----------------------------------------";
148             cout << "\nFarmer ID          : " << farmerId;
149             cout << "\nFarmer Name        : " << name;
150             cout << "\nVillage            : " << village;
151             cout << "\nLand Area          : " << landArea << " acres";
152             cout << "\nCrop Type          : " << cropType;
153             cout << "\nWater Source       : " << waterSource;
154             cout << "\nIrrigation Date    : " << turnDate;
155             cout << "\nStarting Hour      : " << startHour;
156             cout << "\nDuration           : " << durationHours << " hours";
157             cout << "\nStatus             : " << status;
158             cout << "\n----------------------------------------\n";
159         }
160     }
161 
162     friend void showFarmerSummary(const Farmer& f);
163 };
164 
165 int Farmer::farmerCount = 0;
166 
167 void showFarmerSummary(const Farmer& f)
168 {
169     cout << "\nFarmer Summary";
170     cout << "\nID          : " << f.farmerId;
171     cout << "\nName        : " << f.name;
172     cout << "\nVillage     : " << f.village;
173     cout << "\nWater Source: " << f.waterSource << endl;
174 }
175 
176 int main()
```

```
{
    const int MAX = 50;
    Farmer arr[MAX];
    Farmer* ptr = arr;

    int total = 0;
    int choice = 0;
    int id = 0;
    int flag = 0;
    int found = 0;

    for (int i = 0; i < MAX; i++)
    {
        arr[i].setDefaults();
    }

    do
    {
        cout << "\n========== RURAL IRRIGATION TURN ALLOCATION ==========\n";
        cout << "1. Register Farmer\n";
        cout << "2. Display All Farmers\n";
        cout << "3. Search Farmer by ID\n";
        cout << "4. Show Total Registered Farmers\n";
        cout << "5. Exit\n";
        cout << "Enter Your Choice: ";

        while (!(cin >> choice))
        {
            cin.clear();
            cin.ignore(1000, '\n');
            cout << "Enter a valid menu choice: ";
        }

        switch (choice)
        {
        case 1:
            if (total >= MAX)
            {
                cout << "\nMaximum capacity of 50 farmers reached.\n";
                break;
            }

            cout << "\nEnter Farmer ID: ";
            while (!(cin >> id) || id <= 0)
            {
                if (cin.fail())
                {
                    cin.clear();
                    cin.ignore(1000, '\n');
                }
                cout << "Enter a valid positive Farmer ID: ";
            }

            flag = 0;

            for (int i = 0; i < total; i++)
            {
                if (ptr[i].getFarmerId() == id)
                {
                    flag = 1;
                    break;
                }
            }

            if (flag == 1)
            {
                cout << "\nDuplicate Farmer ID! Registration Rejected.\n";
            }
            else
            {
                arr[total].inputDetails(id);

                if (arr[total].isValid())
                {
                    total++;
                }
                else
                {
                    arr[total].setDefaults();
                }
            }
            break;

        case 2:
            if (total == 0)
            {
                cout << "\nNo Farmer Records Available.\n";
            }
            else
            {
                cout << "\n========== ALL REGISTERED FARMERS ==========\n";
                for (int i = 0; i < total; i++)
                {
```

```
                    ptr[i].displayDetails();
                }
            }
            break;

        case 3:
            if (total == 0)
            {
                cout << "\nNo Farmer Records Available to Search.\n";
                break;
            }

            cout << "\nEnter Farmer ID to Search: ";
            while (!(cin >> id) || id <= 0)
            {
                if (cin.fail())
                {
                    cin.clear();
                    cin.ignore(1000, '\n');
                }
                cout << "Enter a valid positive Farmer ID: ";
            }

            found = 0;

            for (int i = 0; i < total; i++)
            {
                if (ptr[i].getFarmerId() == id)
                {
                    showFarmerSummary(ptr[i]);
                    ptr[i].displayDetails();
                    found = 1;
                    break;
                }
            }

            if (found == 0)
            {
                cout << "\nFarmer ID Not Found.\n";
            }
            break;

        case 4:
            cout << "\nTotal Registered Farmers: " << Farmer::getFarmerCount() << endl;
            break;

        case 5:
            cout << "\nThank you for using Rural Irrigation Turn Allocation made by Nishant Kumar!\n";
            break;

        default:
            cout << "\nInvalid Choice. Please Select 1 to 5.\n";
        }

    } while (choice != 5);

    return 0;
}
```

11. OUTPUT:

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\D\MCA\C++ Project> cd "d:\D\MCA\C++ Project\Rural-Irriga
tion-Turn-Allocation\CA1\src\" ; if ($?) { g++ main.cpp -o mai
n } ; if ($?) { .\main }

==============================
 RURAL IRRIGATION TURN ALLOCATION
==============================
1. Register Farmer
2. Display All Farmers
3. Search Farmer by ID
4. Show Total Farmers
5. Exit
Enter your choice: 1
Enter unique Farmer ID: 123
Enter farmer name: Nikku
Enter village name: Dugri
Enter land area in acres: 26.365
Enter crop type: Wheat
Enter water source (Canal/Well): Canal
Enter irrigation date (DD-MM-YYYY): 06-11-2026
Enter starting hour (0-23): 10
Enter duration in hours: 8
Farmer registered successfully!

==============================
 RURAL IRRIGATION TURN ALLOCATION
==============================
1. Register Farmer
2. Display All Farmers
3. Search Farmer by ID
4. Show Total Farmers
5. Exit
Enter your choice: 2

Farmer ID: 123
Name: Nikku
Village: Dugri
Land Area: 26.365 acres
Crop Type: Wheat
Water Source: Canal
Irrigation Date: 06-11-2026
Starting Hour: 10:00
Duration: 8 hours
Status: A

==============================
 RURAL IRRIGATION TURN ALLOCATION
==============================
1. Register Farmer
2. Display All Farmers
3. Search Farmer by ID
4. Show Total Farmers
5. Exit
Enter your choice: 3
Enter Farmer ID to search: 123

Farmer ID: 123
Name: Nikku
Village: Dugri
Land Area: 26.365 acres
Crop Type: Wheat
Water Source: Canal
Irrigation Date: 06-11-2026
```

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\D\MCA\C++ Project> cd "d:\D\MCA\C++ Project\Rural-Irriga
Farmer ID: 123
Name: Nikku
Village: Dugri
Land Area: 26.365 acres
Crop Type: Wheat
Water Source: Canal
Irrigation Date: 06-11-2026
Starting Hour: 10:00
Duration: 8 hours
Status: A

--- Farmer Summary ---
Farmer ID: 123
Name: Nikku
Village: Dugri
Water Source: Canal

==============================
 RURAL IRRIGATION TURN ALLOCATION
==============================
1. Register Farmer
2. Display All Farmers
3. Search Farmer by ID
4. Show Total Farmers
5. Exit
Enter your choice: 4
Total registered farmers: 1

==============================
 RURAL IRRIGATION TURN ALLOCATION
==============================
1. Register Farmer
2. Display All Farmers
3. Search Farmer by ID
4. Show Total Farmers
5. Exit
Enter your choice: 5
Thank you for using Rural Irrigation Turn Allocation made by N
ishant Kumar!
PS D:\D\MCA\C++ Project\Rural-Irrigation-Turn-Allocation\CA1\s
rc>
```